1 - Job Applicants

Now it's time to add the functionality of letting users apply to jobs on the details page. We're going to use Alpine.js to create a modal that opens when we click on the apply button. The user can add fields like their name and contact info but they will also be able to upload a pdf resume.

We'll create a new migration and model for applicants. The job listing owner will see the applicants from the dashboard and will be able to download the resume and delete the applicants. We also will prevent the same user from submitting multiple applications to a single listing.

2 - Applicants Migration and Model

We are now going to start to implement job application submissions. Users can apply for jobs and submit their resumes. We will create a new table to store the applicants and migrate it to the database.

Create Applicants Table

Let's create a new table to store the applicants. We will create a new migration file for this.

```
php artisan make:migration create_applicants_table
```

Open the file `database/migrations/TIMESTAMP_create_applicants_table.php` and add the following code:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('applicants', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->foreignId('job_id')->constrained('job_listings')-
>onDelete('cascade');
            $table->string('full_name');
            $table->string('contact_phone')->nullable();
            $table->string('contact_email');
            $table->text('message')->nullable();
```

```
            $table->string('location')->nullable();
            $table->string('resume_path');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('applicants');
    }
};
```

We have the following columns:

- `job_id`: The ID of the job that the application is for.
- `user_id`: The ID of the user who submitted the application.
- `full_name`: The full name of the applicant.
- `contact_phone`: The contact number of the applicant.
- `contact_email`: The email address of the applicant.
- `message`: A message from the applicant.
- `location`: The location of the applicant.
- `resume_path`: The path to the applicant's resume.
- `created_at`: The timestamp when the applicant was created.
- `updated_at`: The timestamp when the applicant was last updated.

Migrate the Applicants Table

Run the following command to migrate the applicants table to the database:

```
php artisan migrate
```

The `applicants` table will be created in the database.

Create Applicants Model

Let's create a new model for the applicants table. We will create a new model file for this.

```
php artisan make:model Applicant
```

Open the file `app/Models/Applicant.php` and add the following code:

```
<?php
```

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Model;

class Applicant extends Model
{
    use HasFactory;

    protected $fillable = [
        'job_id',
        'user_id',
        'full_name',
        'contact_phone',
        'contact_email',
        'message',
        'location',
        'resume_path',
    ];

    public function job(): BelongsTo
    {
        return $this->belongsTo(Job::class);
    }

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

Make sure that you import the `BelongsTo` class from the `Illuminate\Database\Eloquent\Relations\BelongsTo` namespace.

We have defined the fillable fields and the relationships with the `Job` and `User` models. An applicant belongs to a job and a user.

Job Model

Now we need to add the 1-to-many relationship between the `Job` and `Applicant` models. Open the file `app/Models/Job.php` and add the following method:

```
public function applicants(): HasMany
{
    return $this->hasMany(Applicant::class);
}
```

Import the `HasMany` class from the `Illuminate\Database\Eloquent\Relations\HasMany` namespace.

```
use Illuminate\Database\Eloquent\Relations\HasMany;
```

This method will return all the applicants for a given job.

User Model

Now we need to add the 1-to-many relationship between the `User` and `Applicant` models. It makes more sense to use the word `applications` as the relationship name. Open the file `app/Models/User.php` and add the following method:

```
 public function applications(): HasMany
{
    return $this->hasMany(Applicant::class, 'user_id');
}
```

Now that we have our table, model and relationships, let's work on the form in the next lesson.

3 - Applicant Alpine Form Modal

In this section, we will create a form to allow applicants to submit their job applications. We are going to use Alpine.js to show a form modal.

Open the file `resources/views/jobs/show.blade.php` and find this code:

```
<p class="my-5">
  Put "Job Application" as the subject of your email and attach your resume.
</p>
<a
  href="mailto:{{$job->contact_email}}"
  class="block w-full text-center px-5 py-2.5 shadow-sm rounded border text-base
font-medium cursor-pointer text-indigo-700 bg-indigo-100 hover:bg-indigo-200"
>
  Apply Now
</a>
```

Replace this with the following code:

```
<!-- Applicant Form -->
<div x-data="{ open: false }" id="applicant-form">
  <button
    @click="open = true"
    class="block w-full text-center px-5 py-2.5 mt-5 shadow-sm rounded border
text-base font-medium cursor-pointer text-indigo-700 bg-indigo-100 hover:bg-
indigo-200"
```

```
    >
      Apply Now
    </button>

    <div
      x-show="open"
      class="fixed inset-0 flex items-center justify-center bg-gray-900 bg-opacity-
50"
    >
      <div @click.away="open = false" class="bg-white p-6 rounded-lg shadow-md w-
full max-w-md">
        <h3 class="text-lg font-semibold mb-4">Apply for {{ $job->title }}</h3>

        <form enctype="multipart/form-data">
           <x-inputs.text id="full_name" name="full_name" label="Full Name"
:required="true" />
            <x-inputs.text id="contact_phone" name="contact_phone" label="Contact
Phone" />
            <x-inputs.text id="contact_email" name="contact_email" label="Contact
Email" :required="true" />
            <x-inputs.text-area id="message" name="message" label="Message" />
            <x-inputs.text id="location" name="location" label="Location" />
            <x-inputs.file id="resume" name="resume" label="Upload Your Resume
(pdf)" :required="true" />
            <button
              type="submit"
              class="bg-blue-500 hover:bg-blue-600 text-white px-4 py-2 rounded-md"
            >
              Submit Application
            </button>
            <button
              type="button"
              @click="open = false"
              class="ml-2 bg-gray-300 hover:bg-gray-400 text-black px-4 py-2
rounded-md"
            >
              Cancel
            </button>
        </form>
      </div>
    </div>
</div>
```

Add the `required` prop to the text and file components:

```
@props(['id', 'name', 'label' => null, 'type' => 'text', 'value' => '',
'placeholder' => '', 'required' => false])

<div class="mb-4">
    @if($label)
    <label class="block text-gray-700" for="{{$id}}">{{$label}}</label>
    @endif
```

```
    <input id="{{$id}}" type="{{$type}}" name="{{$name}}"
        class="w-full px-4 py-2 border rounded focus:outline-none @error($name)
border-red-500 @enderror"
        placeholder="{{$placeholder}}" value="{{old($name, $value)}}" {{$required
? 'required' : '' }} />
    @error($name)
    <p class="text-red-500 text-sm mt-1">{{$message}}</p>
    @enderror
</div>
```

```
@props(['id', 'name', 'label' => null, 'required' => false])

<div class="mb-4">
    @if($label)
    <label class="block text-gray-700" for="{{$id}}">{{$label}}</label>
    @endif
    <input {{$required ? 'required' : '' }} id="{{$id}}" type="file" name="
{{$name}}"
        class="w-full px-4 py-2 border rounded focus:outline-none @error($name)
border-red-500 @enderror" />
    @error($name)
    <p class="text-red-500 text-sm mt-1">{{$message}}</p>
    @enderror
</div>
```

We are using Alpine.js to create a modal form for the job application. When the "Apply Now" button is clicked, the form will be displayed as a modal. The form includes fields for the applicant's full name, contact number, contact email, message, location, and resume upload. The form also includes a submit button to submit the application and a cancel button to close the form.

Let's add the action to the form. Update the form tag to include the action attribute and pass in the route and job ID:

```
<form
  action="{{ route('applicants.store', $job->id) }}"
  method="POST"
  enctype="multipart/form-data"
></form>
```

This will raise an error because the route does not exist yet. We will create the route in the next section along with the controller method.

4 - Applicant Controller & Storing Applicants

Now that we have the form to submit the job application, we need to create the controller and routes to handle the form submission and store the data and upload the resume.

Applicant Routes

Let's create the routes for the applicant controller. We need a route to the `store` method, which will handle the form submission.

Open the `web.php` file and add the following import:

``php use App\Http\Controllers\ApplicantController;

```
Now add the route:

```php
Route::post('/jobs/{job}/apply', [ApplicantController::class, 'store'])-
>name('applicants.store');
```

Put it above the job resource routes.

Applicant Controller

Next, let's create the `ApplicantController` and the `store` method to handle the form submission. Open the terminal and run the following command:

```
php artisan make:controller ApplicantController
```

This will create the `ApplicantController` in the `app/Http/Controllers` directory. Open the `ApplicantController.php` file and add the following code:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Job;
use App\Models\Applicant;
use Illuminate\Http\RedirectResponse;

class ApplicantController extends Controller
{
    // @desc   Store a new job application
    // @route  POST /jobs/{job}/apply
    public function store(Request $request, Job $job): RedirectResponse
    {
        // Validate the incoming request data
        $validatedData = $request->validate([
            'full_name' => 'required|string|max:255',
            'contact_number' => 'string|max:20',
            'contact_email' => 'required|email',
```
```

```
            'message' => 'string',
            'location' => 'string|max:255',
            'resume' => 'required|file|mimes:pdf|max:2048',
        ]);

        dd($validatedData);
    }
}
```

Be sure to import the `Job` and `Applicant` models at the top of the file.

Form Submission

I have included a dummy resume.pdf in the files for this section but you can use any pdf. Make sure you have a pdf file ready to submit with the form.

Now submit your applicant form and you should see the data dumped to the screen.

The url should be something like `http://127.0.0.1:8000/jobs/5/apply`

Storing The Resume

Replace the `dd` with the following code:

```
 // Handle the resume file upload
if ($request->hasFile('resume')) {
    $path = $request->file('resume')->store('resumes', 'public');
    $validatedData['resume_path'] = $path;
}
```

We are checking if the resume file is present in the request. If it is, we are storing the file in the `public/resumes` directory and updating the `resume_path` field in the `validatedData` array.

Saving The Applicant

Now let's save the applicant to the database:

```
// Store the application
$application = new Application($validatedData);
$application->job_id = $job->id;
$application->user_id = auth()->id();
$application->save();

return redirect()->back()->with('success', 'Your application has been
submitted!');
```

Here is the full `AppliantController.php` file:

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Job;
use App\Models\Applicant;
use Illuminate\Http\RedirectResponse;

class ApplicantController extends Controller
{
    public function store(Request $request, Job $job): RedirectResponse
    {
        // Validate the incoming request data
        $validatedData = $request->validate([
            'full_name' => 'required|string|max:255',
            'contact_number' => 'string|max:20',
            'contact_email' => 'required|email',
            'message' => 'string',
            'location' => 'string|max:255',
            'resume' => 'required|file|mimes:pdf|max:2048',
        ]);

        // Handle the resume file upload
        if ($request->hasFile('resume')) {
            $path = $request->file('resume')->store('resumes', 'public');
            $validatedData['resume_path'] = $path;
        }

        // Store the application
        $application = new Applicant($validatedData);
        $application->job_id = $job->id;
        $application->user_id = auth()->id();
        $application->save();

        return redirect()->back()->with('success', 'Your application has been
submitted!');
    }
}
```

You should now be able to submit your application and see the data in the database.

Clear Applicants On Seed

You could make it so some applicants are created when you run the seeder, but for now, let's clear the applicants table when we seed the database.

Open thee `DatabaseSeeder.php` file and add the following line under the other truncates:

```
DB::table('applicants')->truncate();
```

Now when you run the seeder, the applicants table will be cleared.

```
php artisan db:seed
```

5 - Show Job Applicants

Now we want to show the applicants to the job owner on their dashboard directly under the job listing.

Update Dashboard Controller

First we need to make a small edit to the `DashboardController` `index` method to get the job applicants.

```
// @desc   Show the dashboard
// @route  GET /dashboard
 public function show(Request $request): View
  {
      // Get the authenticated user
      $user = Auth::user();

      // Get all job listings for the authenticated user
      $jobs = Job::where('user_id', $user->id)->with('applicants')->get();

      return view('dashboard.show', compact('user', 'jobs'));
  }
```

We added `with('applicants')` to the `get()` method to get the applicants for each job.

Show Applicants In View

Now in the `resources/views/dashboard/show.blade.php` file, we need to add the following code in the job listings just above the `@empty`:

```
{{-- Applicants --}}
<div class="mt-4">
  <h4 class="text-lg font-semibold mb-2">Applicants</h4>
  @forelse($job->applicants as $applicant)
  <div class="py-2">
    <p class="text-gray-800">
      <strong>Name: </strong> {{$applicant->full_name}}
    </p>
    <p class="text-gray-800">
      <strong>Phone: </strong> {{$applicant->contact_phone}}
```

```
    </p>
    <p class="text-gray-800">
      <strong>Email: </strong> {{$applicant->contact_email}}
    </p>
    <p class="text-gray-800">
      <strong>Message: </strong> {{$applicant->message}}
    </p>
    <p class="text-gray-800 my-4">
      <a href="{{asset('storage/' . $applicant->resume_path)}}" class="text-blue-
500 hover:underline" download>
        <i class="fas fa-download"></i> Downoad Resume
      </a>
    </p>
  </div>
```

It will show the fields as well as a download link for the resume.

6 - Delete Applicants

Let's add the functionality to delete applicants from the job application submissions.

Open the `/resources/views/dashboard/show.blade.php` file and add the following code right under the `</p>` for the download resume button:

```
<!-- Delete Applicant Link -->
<form
  method="POST"
  action="{{ route('applicants.destroy', $applicant->id) }}"
  onsubmit="return confirm('Are you sure you want to delete this applicant?');"
>
  @csrf @method('DELETE')
  <button type="submit" class="text-red-500 hover:text-red-700 text-sm">
    <i class="fas fa-trash-alt"></i> Delete Applicant
  </button>
</form>
```

You will get an error for now because the `destroy` method is not defined in the `ApplicantController`. Let's define it now. Start by adding the following route in the `web.php` file:

```
Route::delete('/applicants/{applicant}', [ApplicantController::class, 'destroy'])-
>name('applicants.destroy')->middleware('auth');
```

Now add the method to the `ApplicantController`:

```
// @desc   Delete a job application
// @route  DELETE /applicants/{applicant}
```

```
public function destroy($id): RedirectResponse
{
    $applicant = Applicant::findOrFail($id);
    $applicant->delete();
    return redirect()->route('dashboard.show')->with('success', 'Applicant deleted
successfully.');

}
```

Now you should be able to delete applicants from the job application submissions.

7 - Prevent Multiple Applications

I don't want the same user to be able to submit an application to the same job listing. Let's open the `app/Http/controllers/ApplicantController.php` file and change the `store` method to the following:

```
public function store(Request $request, Job $job): RedirectResponse
{
    // Check if the user has already applied for the job
    $existingApplication = Applicant::where('job_id', $job->id)
                                    ->where('user_id', auth()->id())
                                    ->exists();

    if ($existingApplication) {
        return redirect()->back()->with('status', 'You have already applied to
this job.');
    }

    // Validate incoming data
    $validatedData = $request->validate([
        'full_name' => 'required|string',
        'contact_phone' => 'string',
        'contact_email' => 'required|string|email',
        'message' => 'string',
        'location' => 'string',
        'resume' => 'required|file|mimes:pdf|max:2048',
    ]);

    // Handle resume upload
    if ($request->hasFile('resume')) {
        $path = $request->file('resume')->store('resumes', 'public');
        $validatedData['resume_path'] = $path;
    }

    // Store the application
    $application = new Applicant($validatedData);
    $application->job_id = $job->id;
    $application->user_id = auth()->id();
    $application->save();
```

```
        return redirect()->back()->with('success', 'Your application has been
    submitted.');
    }
```

Now a user should only be able to apply once.